



# MSCCL++: Rethinking GPU Communication Abstractions for AI Inference

Changho Hwang  
Microsoft Research  
Vancouver, BC, Canada

Peng Cheng  
Microsoft Research  
Redmond, WA, USA

Roshan Dathathri  
Microsoft Research  
Redmond, WA, USA

Abhinav Jangda  
Microsoft Research  
Redmond, WA, USA

Saeed Maleki\*  
Microsoft Research  
Redmond, WA, USA

Madan Musuvathi  
Microsoft Research  
Redmond, WA, USA

Olli Saarikivi†  
Microsoft Research  
Redmond, WA, USA

Aashaka Shah  
Microsoft Research  
Redmond, WA, USA

Ziyue Yang  
Microsoft Research  
Beijing, China

Binyang Li  
Microsoft Azure  
Redmond, WA, USA

Caio Rocha  
Microsoft Azure  
Redmond, WA, USA

Qinghua Zhou  
Microsoft Azure  
Redmond, WA, USA

Mahdieh Ghazimirsaeed  
Microsoft Azure  
Cambridge, MA, USA

Sreevatsa Anantharamu  
Microsoft Azure  
Minneapolis, MN, USA

Jithin Jose  
Microsoft Azure  
Austin, TX, USA

## Abstract

AI applications increasingly run on fast-evolving, heterogeneous hardware to maximize performance, but general-purpose libraries lag in supporting these features. Performance-minded programmers often build custom communication stacks that are fast but error-prone and non-portable.

This paper introduces **MSCCL++**, a design methodology for developing high-performance, portable communication kernels. It provides (1) a low-level, performance-preserving primitive interface that exposes minimal hardware abstractions while hiding the complexities of synchronization and consistency, (2) a higher-level DSL for application developers to implement workload-specific communication algorithms, and (3) a library of efficient algorithms implementing the standard collective API, enabling adoption by users with minimal expertise.

Compared to state-of-the-art baselines, MSCCL++ achieves geomean speedups of  $1.7\times$  (up to  $5.4\times$ ) for collective communication and  $1.2\times$  (up to  $1.38\times$ ) for AI inference workloads. MSCCL++ is in production of multiple AI services

provided by Microsoft Azure, and has also been adopted by RCCL, the GPU collective communication library maintained by AMD. MSCCL++ is open source and available at <https://github.com/microsoft/mscclpp>. Our two years of experience with MSCCL++ suggests that its abstractions are robust, enabling support for new hardware features, such as multitenancy, within weeks of development.

**CCS Concepts:** • **Software and its engineering** → **Domain specific languages; API languages; Requirements analysis; Abstraction, modeling and modularity; Software performance; Software usability;** • **Networks** → **Programming interfaces.**

**Keywords:** GPU Communication; Collective Communication; AI Inference; Multi-layered Programming Abstraction

## ACM Reference Format:

Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, Caio Rocha, Qinghua Zhou, Mahdieh Ghazimirsaeed, Sreevatsa Anantharamu, and Jithin Jose. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3779212.3790188>

\*Now at xAI.

†Now at Apple.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790188>

## 1 Introduction

GPU communication has emerged as a critical bottleneck and an opportunity for optimization in AI systems. For instance, communication kernels account for approximately 10%-40% in many real-world Large Language Model (LLM)

inference tasks [7, 10, 24]. Recent research has addressed communication bottlenecks, including developing efficient communication algorithms (e.g., routing paths and transfer scheduling) [3, 14, 33], overlapping communication with computation [12, 35], and other lower-level optimizations in the stack [1, 22].

Despite these efforts, achieving high communication performance is still challenging and time-consuming in real applications. A common practice today is for practitioners to write custom communication code from scratch to achieve maximum performance without using standard libraries such as the NVIDIA Collective Communication Library (NCCL) [22]. For instance, NVIDIA's TensorRT-LLM [25] implements custom AllReduce communication kernels rather than relying on or updating NCCL. There are multiple reasons for doing so. First, achieving peak performance requires a careful balance between latency and bandwidth, tailored to the specific message sizes and communication patterns of a workload — something a general-purpose library, designed to support many workloads, cannot always optimize [5]. For example, TensorRT-LLM uses a custom AllReduce that outperforms NCCL on small data sizes, while falling back to NCCL for larger ones. Second, communication collectives often involve computations (e.g., reductions), and their performance depends on exploiting parallelism in ways that interact well with the necessary computation, further motivating workload-specific customization. Third, the enormous computational demand of modern AI applications is driving rapid evolution in both chips and interconnects, and practitioners rush to exploit these new capabilities long before general-purpose libraries such as NCCL are fully adapted.

The practice of writing custom communication code comes at a huge engineering cost. Developing these libraries is challenging and error prone. New hardware capabilities are often immature or poorly documented, forcing developers to navigate low-level details with limited guidance. There are also complex synchronization and data consistency challenges that arise from interactions across GPUs, CPUs, and NICs, which must be carefully managed to avoid correctness issues and performance bottlenecks. Finally, achieving maximum efficiency typically demands the design of custom communication algorithms tailored to the specific workload and hardware, further increasing development complexity.

Our hypothesis to address these issues is that *multi-layered* programming abstractions are needed to provide performance, portability, and productivity at the same time. This paper proposes **MSCCL++**<sup>1</sup>, a design methodology that evaluates this hypothesis for high-performance GPU communication. MSCCL++ provides portable, hierarchical abstractions that enable users to explore the trade-offs between productivity and performance. At its core, MSCCL++ provides a

Primitive API that exposes minimal performance-preserving abstractions of hardware interconnect methods. These abstractions provide full flexibility for customization, including fine-grained overlapping of compute and communication, while hiding the complexities of synchronization and ensuring portability across hardware vendors and generations. Building on the Primitive API, MSCCL++ provides a high-level Domain-Specific Language (DSL) for specifying custom communication algorithms while the MSCCL++ DSL Executor handles implementation concerns and performs optimizations when executing the DSL program using the Primitive API. Using the DSL, MSCCL++ provides a library of collective algorithms optimized for different scenarios that together provide the same API as NCCL. Our design is thus based on a separation of concerns: (1) the Primitive API provides a portable substrate for expert programmers to exploit hardware-specific features for the best performance; (2) the DSL API offers a streamlined interface that allows application developers to implement workload-specific and network-topology-specific optimizations with minimal effort; and (3) the Collective API enables users to leverage these specialized implementations with no additional effort. These three layers offer abstractions that improve programmability while enabling progressive customization layer-by-layer to get the last ounce of performance.

Prior primitive interfaces not only are hardware-vendor-specific but also often mask certain hardware features, limiting opportunities for optimization; e.g., NCCL provides a low-level *send-recv* abstraction that prohibits developers from exploiting one-sided, asynchronous communication over interconnects, and NVSHMEM [23] provides a low-level shared-memory abstraction that prevents interconnect-specific optimizations such as using port-mapped DMA-copy on NVLink. In contrast, the MSCCL++ Primitive API exposes the fundamental building blocks of communication for different data transfer modes supported by hardware interconnects: memory-mapped I/O, port-mapped I/O, and switch-mapped I/O. While the building blocks are specific to the data transfer modes, they are portable across hardware vendors and generations supporting the corresponding data transfer mode. This interface supports one-sided and asynchronous abstractions for efficient communication both within and across nodes. The implementation of these primitives handles orchestration and synchronization across GPU, CPU, and NIC, providing clear data consistency and synchronization semantics.

The MSCCL++ DSL allows developers to specify communication algorithms while abstracting away implementation details. To improve programmability, it provides a thread-block-based API with a global view across all GPUs and exposes the underlying one-sided, asynchronous communication abstractions of the Primitive API. MSCCL++ automatically inserts intra-thread-block synchronizations, fuses

<sup>1</sup>MSCCL++ stands for Microsoft Collective Communication Library ++, pronounced *em-sickle plus-plus*.

instructions, and removes unnecessary memory accesses before lowering to an execution plan that is interpreted by the MSCCL++ DSL Executor. This design accelerates the development of application-specific communication algorithms for new hardware environments. Unlike MSCCLang [5], our DSL retains the one-sided and asynchronous properties of the Primitive API and enables algorithms that overlap communication with computation. Using the DSL, we implement efficient communication algorithms, including those with computation-communication overlap, that are tuned for AI inference workloads. These algorithms are bundled into our library that provides the MSCCL++ Collective API.

In our evaluation, we compare MSCCL++ with NCCL, ROCm Collective Communication Library (RCCL) [1] and Microsoft Collective Communication Library (MSCCL) [4] on NVIDIA A100, NVIDIA H100, and AMD MI300x GPUs. We show that collective communication kernels like AllReduce and AllGather implemented using the MSCCL++ DSL are on average 1.99 $\times$ , 2.08 $\times$ , and 1.43 $\times$  faster than NCCL, RCCL, and MSCCL respectively. We also develop some of these communication algorithms directly using the MSCCL++ Primitive API and show that those kernels are on average 3% faster than their DSL counterparts. These benefits translate to AI inference workloads. LLM inference decodes are on average 1.11 $\times$  and 1.31 $\times$  faster by using MSCCL++ instead of NCCL in vLLM [13] and SGLang [32] respectively.

The MSCCL++ code base [17] has been publicly available for two years. During this time, RCCL [1] has adopted both the MSCCL++ APIs and library as the default to support collective communication primitives on current and upcoming AMD hardware. SGLang [32], a popular LLM inference framework, uses the MSCCL++ library for its collective communication. As a further validation of our design, when NVIDIA introduced support for multitem instructions for switch-mapped I/O (aggregation and multicast), incorporating this support in MSCCL++ (Section 4.3) required only 16 person-weeks, allowing the rest of the MSCCL++ stack to leverage these primitives. Similarly, supporting NVIDIA's multi-node NVLink [20] took only 2 person-weeks.

## 2 Background and Motivation

### 2.1 Collective Communications

Communication in AI workloads often happens through collective operations in which multiple GPUs coordinate to transfer data among themselves and optionally performing computations on the data. *AllReduce* is a common collective communication operation that sums the partial results of all GPUs and broadcasts the computed result to all GPUs. AllReduce can be divided into two other collective operations: *ReduceScatter* and *AllGather*. *ReduceScatter* sums the input buffers on all GPUs and distributes the output buffer equally on all GPUs. *AllGather* collects a distributed buffer from all GPUs and stores the full buffer on each GPU.

### 2.2 Vendor Communication Libraries

GPU vendors provide communication libraries that implement the collective operations. NVIDIA Collective Communication Libraries (NCCL) [22] is an optimized library for collective communications on NVIDIA hardware. Similarly, AMD ROCm Collective Communication Library (RCCL) [1] is optimized for AMD hardware. Microsoft Collective Communication Library (MSCCL) [4] enables executing custom communication algorithm optimized for a particular input size and topology of the hardware. We briefly discuss the design of NCCL (see Hu et al. [9] for a detailed description), since both RCCL and MSCCL are based on NCCL and they both have the same limitations as NCCL.<sup>2</sup>

NCCL implements a GPU kernel for all collective operations. The implementation chooses the communication algorithms (ring, tree, etc.) to implement the collective based on the data size. The core communication happens through four primitive operations: send, recv, copy, and reduce. NCCL allocates send and receive buffers to facilitate the transfer. These operations are synchronous. For instance, calls to send blocks until destination GPUs have called recv to ensure that it is safe to overwrite the send buffer.

### 2.3 Limitations of NCCL Primitives

NCCL primitives are synchronous, which improves programmability, but wastes GPU cycles in busy-wait loops and prevents overlapping of computation and communication. In addition, NCCL's send and recv primitives work over internal send/receive buffers incurring local memory copies. Finally, by being a general-purpose library, NCCL hard codes many performance optimizations providing inflexibility to the users. For instance, NCCL provides limited capability to control the parallelism for performing reduction or copy operations during communication. Similarly, NCCL hardcodes a single transfer mode per link, even though interconnects like PCIe and NVLink support multiple modes of transfer with subtle performance tradeoffs.

## 3 MSCCL++ Overview

MSCCL++ provides three user APIs at hierarchical levels of communication abstractions: Primitive, DSL, and Collective APIs (see Figure 1). The three levels provide different trade-offs in terms of programming effort or expertise and execution latency or performance. The closer the interface is to the hardware, the higher the programmer's control on execution, so higher the performance and programming effort.

**MSCCL++ Primitive API.** This replaces NCCL's primitives with a set of core communication functions that are called from the GPU kernels. This is paired with the bootstrapping API called from the host (CPU) side, which is used for connection setup between GPUs. The primitive interface

<sup>2</sup>RCCL has since adopted code and abstractions from MSCCL++.

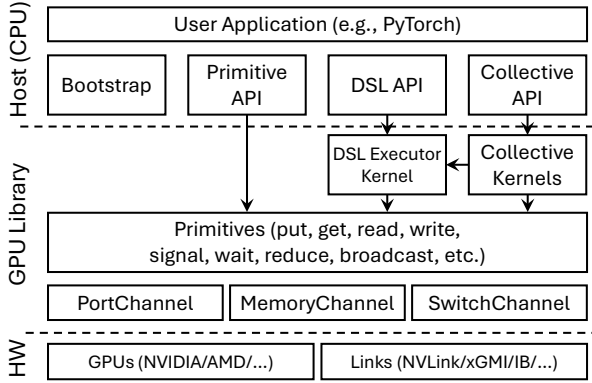


Figure 1. Overview of the MSCCL++ stack.

is the base layer that is used to implement the other two higher-level interfaces. Programmers can also use the primitive interface directly to implement application-specific or hardware-specific optimizations in their own GPU kernels, including those kernels that also perform computation.

**MSCCL++ DSL API.** Based on the primitive interface, we introduce a new DSL to write custom collective communication algorithms in a high-level language. This interface is aimed at users who want to specify communication algorithms or schedules and generate kernels that are optimized for their own workloads (application and hardware). The generated kernel uses the Primitive API by running the DSL Executor, so it may have small runtime overheads.

**MSCCL++ Collective API.** We re-implement the NCCL API, including its bootstrapping API, as-is over MSCCL++. It is aimed at users with the least expertise; they can simply replace NCCL/RCCl with the MSCCL++ Collective library without changing their application code. Similar to NCCL/RCCl, this library may not provide the best algorithm for certain workloads. For AI inference workloads, we implement the best algorithms in our collective kernels using the MSCCL++ DSL.

## 4 MSCCL++ Primitive Abstractions

MSCCL++ Primitive API provides the minimal hardware abstractions for GPU communication. These abstractions are close to the hardware to avoid performance overheads. At the same time, they hide the complexities of GPU/CPU/NIC orchestration, synchronization, and memory consistency issues to simplify the programmability.

The modes of data transfer differ across different interconnects. At its core, MSCCL++ provides the notion of a Channel to capture these modes of communication with basic synchronization primitives. Specifically, MSCCL++ provides (1) `PortChannel` for port-mapped I/O allowing GPUs to initiate data transfer through dedicated hardware I/O ports such as DMA engines or RDMA NICs, (2) `MemoryChannel` for memory-mapped I/O with direct peer-to-peer memory access across GPUs, and (3) `SwitchChannel` for switch-mapped

```

1 class PortChannel
2   void *src, *dst; // set during init
3   uint *semaphore; uint expectedVal;
4
5   void put(ulong dstOff, ulong srcOff, ulong sz);
6   void signal();
7   void wait();
8   void flush();
9
10  template<Protocol protocol> // LL or HB protocol
11  class MemoryChannel
12    void *src, *dst; // set during init
13    uint *semaphore; uint expectedVal;
14
15    void put(ulong dstOff, ulong srcOff, ulong sz,
16            uint tid, uint tids, uint flag=0);
17    template<typename T>
18    T read(ulong off, uint flag=0);
19    template<typename T>
20    void write(ulong off, T elem, uint flag=0);
21    void signal();
22    void wait();
23    void flush();
24
25  class SwitchChannel
26    void *local, *multimem; // set during init
27
28    void reduce(ulong dstOff,ulong srcOff,ulong sz);
29    void broadcast(ulong dstOff,ulong srcOff,
30                  ulong sz);

```

Figure 2. Channel abstractions in MSCCL++ Primitive API.

I/O using *multicasting* memory access enabled by interconnect switches. Each of these channels exposes additional primitives specific to that channel.

An interconnect might support multiple modes of data transfer and hence can support multiple of these interfaces. For instance, NVLink supports all the three, while xGMI and PCIe only support port-mapped and memory-mapped I/O. Our intent is that these abstractions are general allowing the library implementer to quickly adapt to changing hardware capabilities. For instance, the current implementation of `PortChannel` for NVLink requires coordination with the CPU to initiate a DMA engine (`cudaMemcpy`), but can directly initiate it from the GPU if future hardware supports it.

Port-mapped, memory-mapped, and switch-mapped I/O are complementary I/O methods in general computer architectures. `PortChannel`, `MemoryChannel`, and `SwitchChannel` are our abstractions to provide these I/O methods from inside GPU kernels. We believe that they are flexible enough to accommodate future hardware advances. For example, if future hardware interconnects support initiating DMA-copy from within a GPU kernel [11], the same `PortChannel` API can target the new hardware.

Figure 2 shows three types of channels and its interface in MSCCL++. MSCCL++ also supports a few *fused* primitives (omitted in Figure 2 for brevity) that can reduce the overhead of API calls. For example, since a `put` is usually followed by a `signal`, we provide the `put_with_signal` function that performs both at once. The buffers and semaphores are allocated on the GPU memory before initialization of the channels. Next, we describe the three kinds of MSCCL++ channels.

#### 4.1 PortChannel

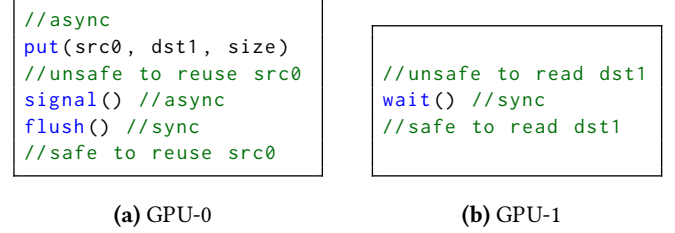
A PortChannel implements primitives when data transfer is done over ports connected to GPU memory, such as `cudaMemcpy` for intra-node DMA or `ibv_post_send` for RDMA. Borrowing the term from MPI [34], writing data to the peer’s side is called **put**. To provide our communication abstraction as close as possible to hardware capabilities, we design `put` to be zero-copy (i.e., no intermediate buffers), one-sided (i.e., initiated by a peer without participation of the other peer), and asynchronous.

Figure 3 shows the semantics of the synchronization primitives: `signal`, `wait`, and `flush`. The `put` primitive transfers data from `src0` buffer of GPU-0 to `dst1` buffer of GPU-1. The `signal` is an asynchronous operation that is strictly ordered with respect to the previous transfer operations, such as `put` here. The matching `wait` primitive called by GPU-1 blocks till it receives GPU-0’s `signal` at which time GPU-1 is allowed to access the transferred data in `dst1`. GPU-0 calls the `flush` primitive to ensure that the transfer is complete from its perspective allowing it to reuse the `src0` buffer.

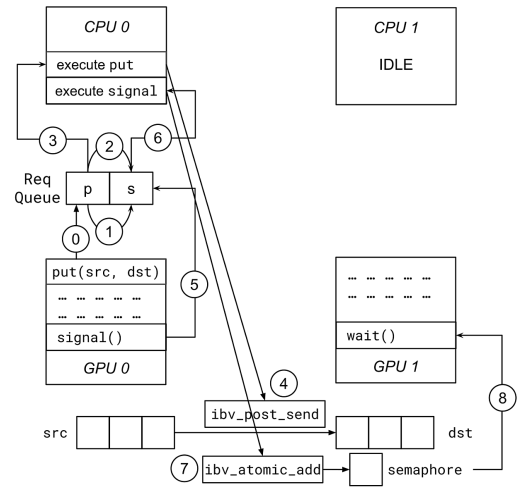
**Implementation.** Since data transfer over a port currently requires the CPU to initiate the transfer, each channel creates its own CPU thread. The CPU thread reads data transfer or synchronization requests from a first-in-first-out request queue. The storage and tail of a request queue are allocated using `cudaMallocManaged` as both CPU and GPU can access it, while the head is on GPU and only accessed by GPU.

We now discuss the workflow of PortChannels using Figure 4. ① When the GPU calls a primitive, such as `put`, the first participating thread of the GPU pushes the request to the queue by writing at the head. Before writing to the queue, the GPU checks if the queue is filled, i.e., if the head value is more than the tail value. If the queue is filled then the GPU waits for the CPU to process at least one request. ② Then the GPU increments the head to the next element. ③ The CPU thread continuously reads the element at the tail to see if there is a request from the GPU, and when there is a request, the CPU reads the request, zeros out the current element, and increments the tail. ④ Now the CPU thread will process the request. Below we explain three types of requests and how the CPU handles them over InfiniBand (IB) as an example.

**Data Transfer.** `put` pushes a `put` request in the queue. ④ The CPU processes this request by starting an RDMA transfer using `ibv_post_send`. Since this function is asynchronous,



**Figure 3.** MSCCL++ data transfer abstractions. `put` asynchronously transfers data from one GPU to another. `signal` and `wait` synchronize data transfer between GPUs. `flush` ensures the completion of preceded data transfer.



**Figure 4.** PortChannel workflow for IB from ① when GPU 0 calls `put` primitive to ⑧ when GPU 1 receives the data.

the CPU thread immediately returns. When the transfer is happening, peer-GPUs are free to execute code, thereby, improving the overall power efficiency of the system.

**Synchronization.** ⑤ `signal` pushes a `signal` request in the queue. ⑥⑦ The CPU processes this request by atomically incrementing the semaphore on the receiving GPU using functions, like `ibv_atomic_add` for IB. The `wait` primitive on the receiving GPU does not create a request for its CPU, therefore, the receiving CPU is idle, rather `wait` continuously looks for the semaphore to reach an expected value in a while loop. ⑧ `wait` returns after the semaphore is incremented.

**Flush.** Primitive `flush` pushes a `flush` request in the queue and the first thread of the GPU waits until the queue tail reaches the position where the `flush` request is pushed to. The CPU processes the `flush` request by waiting until all the previous data transfer and synchronization requests have been completed. For example, for IB we use `ibv_poll_cq` to get the status of all requests. After the `flush` request is complete, the GPU is free to re-write to the source buffer.

## 4.2 MemoryChannel

A `MemoryChannel` wraps data transfer methods that use thread-copy mechanism, i.e., directly use GPU threads for writing to peer GPU's memory.

**Protocols.** The channel provides two protocols, which define data transfer and synchronization technique to tune between low-latency and high-bandwidth: *HB* protocol provides a high-bandwidth but high-latency protocol, thus, is suitable for larger sizes, and *LL* protocol provides low-latency but low-bandwidth, thus, is suitable for smaller sizes. Both protocols achieve these properties by synchronizing on the suitable granularity of data.

(i) In the *HB* protocol, peer GPUs transfer a large chunk of data and synchronize this chunk once using `wait` and `signal` primitives. Hence, the synchronization time is amortized over the transfer time leading to high bandwidth. However, since the receiving GPU needs to wait for the whole chunk before processing it, the *HB* protocol also has high latency.

(ii) In *LL* protocol, the receiving GPU synchronizes on fewer chunks of data being transferred and processes the chunk as soon as it is received, thus achieving lower latency. However, since the number of synchronization is proportional to the number of elements, this protocol provides lower-bandwidth. The *LL* protocol works as follows. The `put` primitive requires an extra integer `flag` value. For every  $N - 1$  elements written to the receiving GPU, `put` also writes the `flag`. The receiving GPU uses the `read` primitive, waits until the `flag` value at  $N$  index of receiving buffer is set, and then reads and returns the  $N - 1$  elements. However, we cannot arbitrarily use any  $N$  because GPUs follow a weak memory consistency model in which writes to different memory locations by multiple threads can be performed in any order. Therefore, we restrict  $N$  to number of elements written by a single instruction, i.e., 4, 8, and 16 bytes memory accesses instructions. The `flag` value is decided based on the algorithm, such that, all `flag` values are distinct.

**Data Transfer.** `put` is a collective-thread operations that reads data elements from source buffer and writes to destination buffer starting from different offsets using a set of threads. Similarly, the `read` primitive reads and returns data from the destination buffer, and the `write` primitive writes data including the `flag` for the *LL* protocol to the destination buffer. To maximize bandwidth for the *HB* protocol, both primitives use 16-bytes loads and stores. For the *LL* protocol, both primitives also take a `flag` and use 8-byte loads and stores by default or user supplied vector length. Since both `put` primitives are called by multiple threads (all or first few threads of the kernel), we achieve maximum bandwidth.

**Synchronization.** Synchronization primitives wait on a semaphore that is an integer allocated on the receiving GPU. The `signal` primitive atomically increments the receiving GPU's semaphore and calls `threadfence_system` to ensure

that writes by `put` and semaphore increments are made available in this order. The `wait` primitive performs a busy-wait while-loop that checks until the value of semaphore has reached the expected value. This wait is performed by the first thread of the kernel and all other threads wait on a kernel barrier. The channel tracks the expected value using its `expectedValue` member. The `flush` primitive is a no-op because after `put` returns, the source buffer can be reused, even though the write is still in progress.

## 4.3 SwitchChannel

A `SwitchChannel` provides two primitives for performing collective operations among GPUs: `reduce` to add the corresponding elements of buffers residing on different GPUs and `broadcast` to send elements from a buffer on a GPU to all other GPUs. These operations usually require specialized hardware support. For example, NVIDIA NVLink 4.0 connects all H100 GPUs to a single NVSwitch and this NVSwitch can perform in-network, switch-based aggregation and multicast using NVLink SHARP (NVLS) technology [19]. We now discuss the implementation of these primitives for NVSwitch, but we believe that these implementations can be generalized to other hardware in the future.

**Reduce.** This primitive takes a destination buffer as a local address on the local GPU and a source buffer allocated as a *multimem* address. A *multimem* address is a virtual address that points to different virtual addresses on each GPU that is a part of the channel/collective. The primitive goes through each element of the destination buffer on the local GPU and executes `multimem.ld_reduce` PTX instruction using the source element's *multimem* address. The *multimem* instruction fetches the values from all the virtual memory addresses pointed by the *multimem* address to the switch, does the reduction on the switch, and returns the reduced value to the local GPU. The reduced output is obtained in a register and then written to the destination buffer.

**Broadcast.** This primitive takes a source buffer as a local address on the local GPU and a destination buffer as a *multimem* address. The primitive goes through each element of the source buffer on the local GPU, reads the element into a register, and executes a `multimem.st` PTX instruction using the source register and the destination element's *multimem* address. The *multimem* instruction sends the register value to the switch, which broadcasts and stores the value to all virtual memory addresses pointed by the *multimem* address.

## 4.4 Advantages of Primitive API

Expert developers can use the Primitive API to build custom workload-specific communication algorithms in a portable manner with minimal performance overhead (see Table 1). At the same time, developers implementing these primitives can add support for emerging hardware and interconnects. By separating primitives for data transfer and synchronization, MSCCL++ allows asynchronous communication and

|                               | Best Achievable | MSCCL++ |
|-------------------------------|-----------------|---------|
| NVLink Throughput (GB/s)      | 397.5           | 397.5   |
| NVLink Latency (ns)           | 822             | 829     |
| InfiniBand Throughput (GB/s)  | 48.94           | 48.94   |
| InfiniBand Latency ( $\mu$ s) | 3.76            | 4.89    |

**Table 1.** MSCCL++ Primitive API’s peer-to-peer performance compared to the best achievable on our H100 environment (see Table 2). The best achievable performance is measured by nvbandwidth [21] and RDMA perf test [31]. MSCCL++’s NVLink throughput and latency are measured with PortChannel and MemoryChannel, respectively. MSCCL++’s InfiniBand throughput and latency are measured with PortChannel.

batching synchronization of multiple data transfers. By using channels for specific GPU interconnects, MSCCL++ allows users to specialize kernels for particular interconnects. Furthermore, MSCCL++ avoids extra copies in kernels, leading to a kernel with less code paths and zero register spills. Since the Primitive API is an in-kernel interface for communication, we can co-optimize both computation and the communication kernels, leading to better application performance. One example is overlapping reduction in AllReduce with communication (see Section 5.2).

## 5 MSCCL++ DSL

To enable specifying custom communication algorithms, MSCCL++ supports a Python-based Domain-Specific Language (DSL) and lowers the algorithm into a sequence of operations that can be executed by the DSL Executor on top of the primitive abstractions described above. The primitives exposed at the DSL retain the zero-copy, one-sided, and asynchronous properties of the underlying primitive abstractions, thereby enabling users to specify optimized algorithms that overlap compute and communication.

### 5.1 DSL

MSCCL++ DSL provides a global view of all thread blocks on all GPUs or ranks in a Python-native language. As shown in Figure 5, the DSL exposes PortChannel, MemoryChannel, and SwitchChannel abstractions in this global view. Consequently, unlike the Primitive API (Section 4), users specify the ranks involved in each channel and for each operation in the channel, specify the thread block on the channel’s source rank that must execute it. Some operations on the MemoryChannel also accept an optional ThreadBlockGroup that enables a group of thread blocks to collectively perform that operation. The Channel classes include a few *fused* primitives (omitted in Figure 5 for brevity) that can reduce the overhead of API calls. For example, MemoryChannel’s `reduce_put` reduces two local buffers and puts the result to a remote destination using a temporary register to avoid

```

1 class PortChannel:
2     def __init__(self, dst_rank: int,
3                 src_rank: int): ...
4     def put(self, dst_chunk: Buffer,
5            src_chunk: Buffer, tb: int): ...
6     def signal(self, tb: int): ...
7     def wait(self, tb: int): ...
8     def flush(self, tb: int): ...
9
10 class MemoryChannel:
11     def __init__(self, dst_rank: int,
12                  src_rank: int): ...
13     def put(self, dst_chunk: Buffer,
14            src_chunk: Buffer, tb: int,
15            tb_group: ThreadBlockGroup): ...
16     def put_packets(self, dst_chunk: Buffer,
17                    src_chunk: Buffer, tb: int,
18                    tb_group: ThreadBlockGroup): ...
19     def reduce(self, dst_chunk: Buffer,
20               src_chunk: Buffer, reduce_op: ReduceOp,
21               tb: int, tb_group: ThreadBlockGroup): ...
22     def signal(self, tb: int): ...
23     def wait(self, tb: int): ...
24
25 class SwitchChannel:
26     def __init__(self, ranks: List[int]): ...
27     def broadcast(self,
28                  dst_multimem_chunk: MultimemBuffer,
29                  src_chunk: Buffer, tb: int): ...
30     def reduce(self, dst_chunk: Buffer,
31               src_multimem_chunk: MultimemBuffer,
32               reduce_op: ReduceOp, tb: int): ...

```

**Figure 5.** Channel abstractions in MSCCL++ DSL (Python).

a memory round-trip for the intermediate data. In addition, the DSL provides a separate Buffer abstraction and the Channels are not tied to a pair of buffers, so users specify the chunks of the source and destination buffer in each data transfer operations. Since the DSL is Python-native, the chunks are specified as slices of Buffer. Buffers are associated with a rank and allow computation like `reduce`, which is performed on its rank. The DSL also provides convenience classes to create input and output buffers based on the communication pattern (e.g., AllReduce, AllGather, ReduceScatter), device-wide synchronization primitives, and multi-device synchronizations primitives; synchronization within a thread block is handled automatically by the DSL. Users can control chunk sizes, number of queue pairs (QPs), etc. via the DSL. For instance, if users want two QPs, they can use two parallel PortChannels. If users want to offload the communication to CPU, they can use PortChannel rather than MemoryChannel. Thus, the MSCCL++ DSL enables users to construct complex data movement and synchronization workflows without writing low-level CUDA code.

```

1 def ringRS(portChannels, bufs, sz, N, tb=0):
2   scr = [Buffer(rank, sz) for rank in range(N)]
3   for rank in range(N):
4     putChan = portChannels[(rank+1)%N]
5     recvChan = portChannels[(rank+N-1)%N]
6     src = bufs[rank]
7     recv, dst = scr[rank], scr[(rank+1)%N]
8
9   def chunk(step):
10    csz = sz / N
11    offset = ((rank + N - step) % N) * csz
12    return offset, offset+csz/2, offset+csz
13
14   for step in range(N):
15     beg, mid, end = chunk(step)
16     # (a) Put 1st half of chunk
17     putChan.put(dst[beg:mid], src[beg:mid], tb)
18     putChan.signal(tb)
19     if step != 0:
20       pbeg, pmid, pend = chunk(step-1)
21       # Reduce prev 2nd half, overlaps (a)
22       src[pmid:pend].reduce(recv[pmid:pend], tb)
23     recvChan.wait(tb) # Wait for 1st half
24     putChan.flush(tb) # Flush 1st half
25     # (b) Put 2nd half of chunk
26     putChan.put(dst[mid:end], src[mid:end], tb)
27     putChan.signal(tb)
28     # Reduce 1st half, overlaps (b)
29     src[beg:mid].reduce(recv[beg:mid], tb)
30     recvChan.wait(tb) # Wait for 2nd half
31     putChan.flush(tb) # Flush 2nd half
32     if step == N-1:
33       src[mid:end].reduce(recv[mid:end], tb)

```

**Figure 6.** A simplified example of a Ring ReduceScatter implementation using MSCCL++ DSL.

## 5.2 Example: Overlapped Ring ReduceScatter

Figure 6 demonstrates how the DSL can implement an efficient Ring ReduceScatter algorithm that overlaps computation and communication. In the classic Ring algorithm,  $N$  GPUs are organized in a ring, the input buffer is divided into  $N$  chunks, and each chunk circulates around the ring to be reduced. At each step, a GPU receives a chunk, reduces it locally, and then forwards it to the next GPU, with all steps occurring synchronously. To enable overlap, Figure 6 further splits each chunk into two halves. This allows the reduction of the first half (Line 29) to overlap with the communication of the second half (Line 26), and the reduction of the second half (Line 22) to overlap with the communication of the first half (Line 17). In Figure 6, only thread block 0 on each GPU does this computation and communication. The user can experiment with more parallelism or pipelining by using more thread blocks on each GPU and/or using more chunks per GPU. The MSCCL++ DSL supports replicating program

instances to increase parallelism and improve performance while automatically handling mapping of the program to thread blocks and channels.

## 5.3 Lowering and Optimizations

The program written in MSCCL++ DSL is lowered to an *execution plan*, which consists of information from the program such as the collective, channel types, data transfer protocol, memory buffers to register for each thread block, semaphores to setup, and the sequence of operations (which may include loops) to run in each thread block of every rank. Buffer sizes and rank counts ( $N$  in Figure 6) are provided by the user at lowering time. During lowering, the program goes through data dependence analysis to add appropriate synchronizations, as well as optimization passes such as operation fusion. This enables the DSL to achieve performance comparable to hand-tuned CUDA implementations.

**Data Dependence Analysis.** MSCCL++ DSL automatically tracks data dependencies at the chunk level within each thread block by maintaining the last writer and active readers for each memory slot. When operations have data dependencies, the lowered program includes necessary synchronization points to ensure correct execution order. For example, in Figure 6, it introduces a thread block synchronization on thread block 0 before the reduce operation (Line 29). The DSL also detects redundant synchronizations and removes them. Specifically, if a lowered program will be calling multiple thread block synchronizations back-to-back, the redundancies will be removed, retaining only one of them.

**Operation Fusion.** MSCCL++ DSL also maintains a directed acyclic graph (DAG) to track data dependencies and usage patterns of operations in a thread block at the chunk level. When two or more operations meet the fusion criteria, such as contiguous chunk access, no intervening dependencies, and compatible resource requirements, the DSL merges them into a single operation function. This fusion strategy reduces memory traffic and avoids unnecessary synchronization, resulting in faster execution. For example, a DSL code using `MemoryChannel`:

```
src.reduce(data, tb); memChan.put(dst, src, tb);
```

is captured into a fused operation:

```
memChan.reduce_put(dst, src, data, tb);
```

where `reduce_put` reduces two local buffers and puts the result to a remote destination using a temporary register to avoid a memory round-trip for the intermediate data.

## 5.4 Executor

Given an execution plan and the input and output buffers, the DSL Executor initializes different channels, setting up connections and semaphores, and registering memory. Finally, the executor runs an execution kernel that interprets the list of operations in the execution plan. The execution kernel is

| Env. Name | GPU                         | Intra-node Link       | Network                                         |
|-----------|-----------------------------|-----------------------|-------------------------------------------------|
| A100-40G  | NVIDIA A100 (40G) (8x/node) | NVLink 3.0            | Mellanox HDR InfiniBand (200 Gb/s, 1x NIC/GPU)  |
| A100-80G  | NVIDIA A100 (80G) (8x/node) | NVLink 3.0            | Mellanox HDR InfiniBand (200 Gb/s, 1x NIC/GPU)  |
| H100      | NVIDIA H100 (8x/node)       | NVLink 4.0            | Quantum-2 CX7 InfiniBand (400 Gb/s, 1x NIC/GPU) |
| MI300x    | AMD MI300x (8x/node)        | Infinity Fabric Gen 4 | Quantum-2 CX7 InfiniBand (400 Gb/s, 1x NIC/GPU) |

**Table 2.** List of environments used for evaluation.

a single GPU kernel implementation that runs any given execution plan. The execution kernel inlines calls to primitive MSCCL++ operations such as `put`, `signal`, `wait`, and `flush`. The execution plan can also contain fused operations, such as `ReduceSend` explained in Section 5.3.

## 6 MSCCL++ Collectives Library

MSCCL++ Collective API is the simplest API that MSCCL++ offers, and it is the same as the popular NCCL or RCCL API. We implement this API using kernels written in the MSCCL++ DSL. Users can either use these default kernels or plug-in their own algorithms written using the MSCCL++ DSL or Primitive APIs. We now briefly describe a few AllReduce algorithms included in the default kernels to: (i) provide ideas on low-level considerations for implementing high-performance GPU communication kernels, and (ii) to show how low-level interfaces can be useful for customization on different workloads.

**1. One-phase All-pairs (1PA).** In *all-pairs* algorithms, all GPUs concurrently broadcast their own local data to all other GPUs. By *one-phase*, the algorithm sends all local data to all other GPUs, so that the reduction is done in a single phase. It is suitable for small message sizes where the synchronization overhead between GPUs is more critical than the redundant reduction and data traffic. As we use the 1PA algorithm only for very small message collectives within a single node, we implement only a single version of 1PA that uses the LL protocol with `MemoryChannel`. MSCCL++ can implement it much more efficiently than other libraries by relaxing synchronizations and concurrently transferring data to multiple devices before waiting on any of them.

**2. Two-phase All-pairs (2PA).** By *two-phase*, the algorithm splits the AllReduce into two phases: the first for `ReduceScatter` (each of  $N$  GPUs collects and reduces  $1/N$  of the data) and the second for `AllGather` (each GPU broadcasts the reduced data to all other GPUs) [30]. Two-phase algorithms are more bandwidth-efficient and conduct less reduction than one-phase algorithms. In 2PA, the `ReduceScatter` and `AllGather` phases are done in the all-pairs manner each. We use 2PA for single-node collectives and implement multiple variants using `PortChannel`, `MemoryChannel` with LL or HB protocol, or `SwitchChannel`. Using MSCCL++, we can optimize 2PA in various ways that are not possible in existing libraries. For example, for up to a few MBs of messages, we exploit rotating buffers to reduce synchronization at the cost of using more memory space. As another example, we can

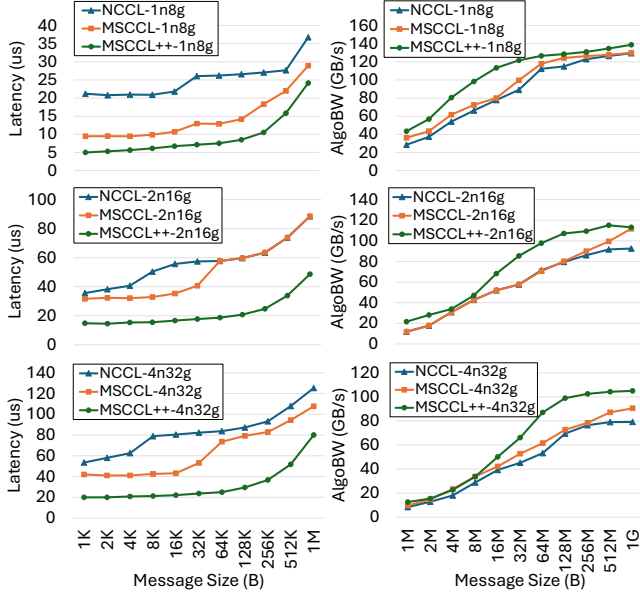
let a single thread group read data from multiple other GPUs at the same time. This allows efficient data reduction compared with other libraries that read data from different GPUs one-by-one, which synchronizes at each reduction step.

**3. Two-phase Ring (2PR).** Similar to 2PA, 2PR has two phases: the first phase does `ReduceScatter` in a ring and the second phase does `AllGather` in the same ring. Unlike NCCL, we can use `PortChannel` (DMA-copy) even within a node and we overlap the reduction with the DMA-copy. Figure 6 shows a simplified kernel for `ReduceScatter`. We extend such pipelining to overlap reduction with communication across both phases. Our 2PR implementation using `PortChannel` shows the best throughput among all implementations for intra-node AllReduce with large message sizes.

**4. Two-phase Hierarchical (2PH).** Hierarchical algorithms exchange minimal data across nodes and do local collectives in each node to complete the operation. It can be faster than all-pairs by reducing the data traffic crossing the nodes. 2PH is a two-phase algorithm that performs `ReduceScatter` and `AllGather` in a hierarchical manner each. We use it for multi-node collectives and implement two variants. The first version is for small messages using the LL protocol. Each node conducts local `ReduceScatter` that splits the data into the number of GPUs in a node. This requires sending more data across nodes and introduces redundant reduction, but it is faster for small messages by reducing synchronization steps. Cross-node communication is done in all-pairs manner. To utilize inter-/intra-node links at the same time, we pipeline the local collective with cross-node communication to overlap. The second version of 2PH is for large messages using HB protocol. This variant also performs all-pairs cross-node communication and local `ReduceScatter` in a pipelined manner. Unlike the first variant, to utilize the link bandwidth efficiently, the number of data chunks is the same as the number of GPUs.

## 7 Evaluation

**Environments.** Table 2 lists the environments used in the evaluation. Each node is equipped with 8 GPUs per node (either A100, H100, or MI300x) with intra-node links between GPUs (either NVLink or Infinity Fabric (a.k.a. xGMI)) and inter-node IB links (one NIC per GPU). All NICs are connected to a single IB networking switch. NVIDIA GPUs use CUDA 12.4, while AMD GPUs use ROCm 6.2. For brevity, the rest of this paper refers to each environment by the name of GPU.



**Figure 7.** AllReduce, A100-40G.  $MnNg$  means  $M$  nodes and  $N$  GPUs. X-axis is message size (at the bottom of the figure).

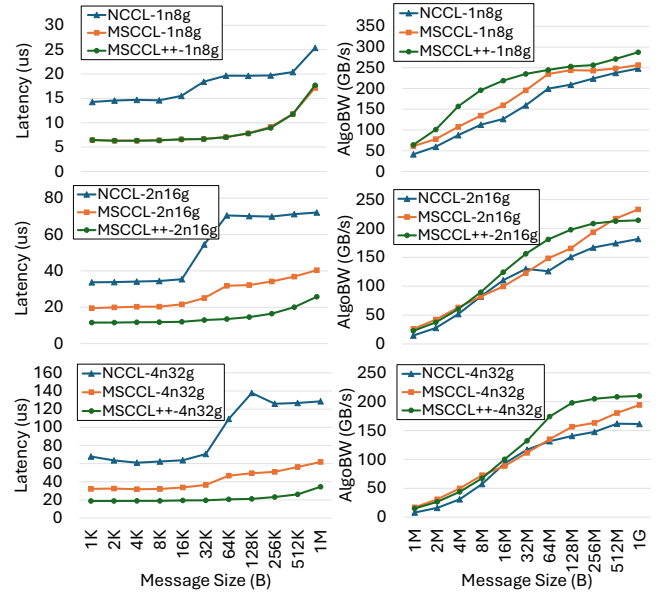
**Baselines.** We compare MSCCL++ with the state-of-the-art collective communication libraries, including NCCL 2.26.2 [22], RCCL 2.20.5 [1], and MSCCL 2.23 [16].

### 7.1 Collective Communication Results

To compare the performance of collective communication, we implement the AllReduce algorithms described in Section 6 using the MSCCL++ DSL. We also implement various AllGather algorithms in a similar manner, and we omit the implementation details. We present the best number among all implementations for each message size and environment<sup>3</sup>. All NCCL, RCCL, and MSCCL numbers are fine-tuned for each environment and message size by adjusting their environment variables, such as the number of channels (affects the number of threads), chunk size (affects the size of data to be transferred at once), type of algorithm (such as ring, tree, or NVLS [19]), the topology (XML file that describes the intra-node link topology of GPUs), etc. For MSCCL, we use the fastest algorithm for each message size [18]. We use NCCL’s user buffer registration API (i.e., `ncclMemAlloc`) [26] and the CUDA/HIP Graph APIs for best performance.

For clarity of exposition, we separate message sizes into small (up to 1MB, presented as latency) and large (1MB and above, presented as algorithm bandwidth (AlgoBW)) in the figures. The small message sizes represent inference scenarios (such as LLM token sampling a.k.a. *decode* [29]),

<sup>3</sup>We did not do sophisticated autotuning beyond picking the best configuration for a given platform and input size ranges (akin to MSCCLang [5]) by offline profiling. Our experiments measure the performance of the resulting system. Orthogonal techniques from TACCL [33] and TE-CCL [14] can be used for automation, which is an interesting future direction.



**Figure 8.** AllGather, A100-40G.  $MnNg$  means  $M$  nodes and  $N$  GPUs. X-axis is message size (at the bottom of the figure).

while large message sizes represent both training and inference scenarios (such as gradient accumulation during back-propagation, or LLM prompt processing a.k.a. *prefill* [29]).

**Results.** Figure 7 compares the AllReduce performance over A100-40G. MSCCL++ outperforms baselines for both small messages (up to **4.2x** and **3.1x** faster over NCCL and MSCCL, respectively) and large messages (up to **1.8x** faster over both NCCL and MSCCL). Similarly, Figure 8 compares the AllGather performance over A100-40G. For small messages, MSCCL++ outperforms NCCL and MSCCL by up to **5.4x** and **2.3x**, respectively, and for large messages, MSCCL++ outperforms NCCL and MSCCL by up to **1.8x** and **1.4x**, respectively. MSCCL++ is faster than other baselines across the board for AllReduce. For AllGather, MSCCL outperforms MSCCL++ by 8.0% in one scenario. This is due to the performance overhead of our DSL API that we are able to reduce if we implement directly on top of MSCCL++’s Primitive API.

**Gain Breakdown.** The performance gain of MSCCL++ comes from both algorithmic improvements, fine-grained overlapping of compute and communication, and the ability to exploit low-level optimization capabilities provided by the Primitive API.

**NCCL vs MSCCL.** MSCCL is faster than NCCL in most cases, and all the benefit comes from better algorithms customized for our environments. The gain is especially big for small message sizes, where MSCCL uses an all-pairs algorithm (which is not supported by NCCL), while NCCL uses the ring algorithm that is worse in terms of latency. For very large message sizes  $\geq 256$ MB in multi-node cases, MSCCL achieves higher bandwidth than NCCL because it uses a hierarchical algorithm (2PH), which is more bandwidth-efficient.

**MSCCL vs MSCCL++.** MSCCL++ provides additional benefits over MSCCL by enabling more efficient variants of those algorithms. For example, for single-node 1KB - 16KB messages in Figure 7, both MSCCL and MSCCL++ use the 1PA algorithm. For 1KB, we observe that MSCCL++ cuts the latency by 47% (from 9.5 $\mu$ s to 5.0 $\mu$ s), showing that the minimum overhead of the communication stack is substantially reduced. Larger messages from 32KB in single-node use various versions of the 2PA algorithm. Especially from 1MB to larger, efficient bandwidth utilization starts to matter, and MSCCL++ shows better scalability than MSCCL. For 1GB in single-node, MSCCL++ uses PortChannel that is not supported by NCCL/MSCCL within a single node. In this case, we observe that PortChannel achieves 6.2% higher bandwidth than MemoryChannel. For 2-node and 4-node results in Figure 7, the 2PH algorithm is used in all cases. MSCCL++ shows substantial gains for both small and large messages.

**DSL vs Primitive.** From our 9-month experience using the MSCCL++ DSL API, we find that it reduces development time from weeks to days, compared to using the Primitive API directly. The DSL helps automate initialization, check for mistakes, and provide performance analysis. We have implemented both the DSL and Primitive versions up to 2-node experiments to verify that their performance is similar, while 4-node experiments are done only using the DSL. As the DSL introduces a runtime interpreter executor, DSL versions perform 3% worse than the Primitive versions on average, and are up to 18% worse in one corner case. Despite performance overheads, MSCCL++ DSL is useful for quick prototyping of collective communication with easy-to-understand algorithm description.

## 7.2 Cross-Hardware Support

To show the effectiveness of MSCCL++ on different hardware platforms, Figure 9 and 10 compare the single-node AllReduce performance on H100 and MI300x.

**H100.** MSCCL++ outperforms NCCL and MSCCL by up to 2.8x and 1.6x for small messages, and by up to 2.4x and 2.0x for large messages, respectively. Our 2PA implementation made efficient use of H100’s NVSwitch via the SwitchChannel interface, which delivers most of the benefit shown for large message sizes. Due to hardware acceleration, we observe up to 56% higher bandwidth by using SwitchChannel compared with an equivalent MemoryChannel implementation. In particular, our SwitchChannel code is only **15 lines of Python code** using our DSL, which simply calls reduce and broadcast element-wise in a loop. This shows that MSCCL++ offers simple yet efficient interfaces, and a lot of unnecessary overhead in existing stacks can be avoided by using MSCCL++.

**MI300x.** MSCCL++ outperforms RCCL and MSCCL by up to 3.8x and 1.9x for small messages, and by up to 2.2x and 1.6x for large messages, respectively. Our algorithm implementations take special considerations to best utilize the Infinity

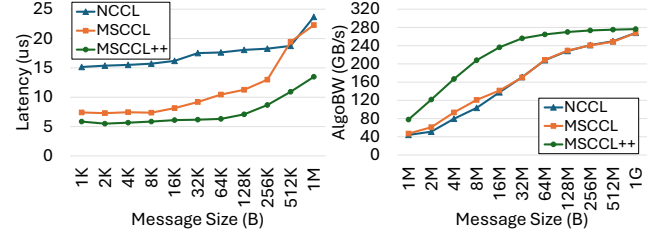


Figure 9. AllReduce, H100, single-node (8 GPUs).

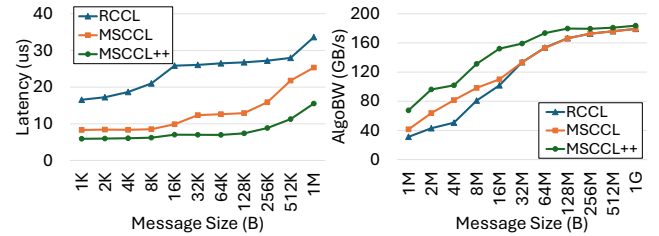


Figure 10. AllReduce, MI300x, single-node (8 GPUs).

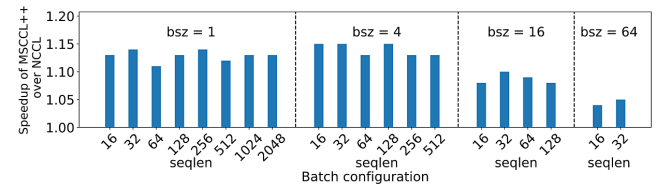
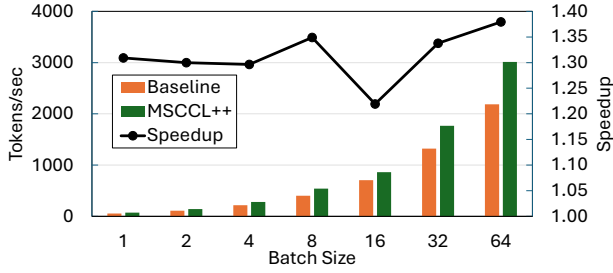


Figure 11. Speedup of MSCCL++ over NCCL for Llama3-70b decodes with tensor parallelism = 8.

Fabric that peer-to-peer connects all GPUs in a node, unlike NVLink that connects all GPUs to a switch. This means, for best link utilization, we need to copy data to all peers at the same time as much as possible, unlike for NVIDIA GPUs where we can copy data to each peer one-by-one back-to-back. Using the MSCCL++ DSL, this is as easy as changing the order of two nested for loops, one that walks through data elements and the other that walks through peers.

## 7.3 LLM Inference

**LLM Inference with vLLM.** We evaluate the benefit of MSCCL++ in end-to-end distributed inference of a popular LLM, Llama3-70b [8] using the LLM inference library, vLLM [13] (v0.3.3). We modified vLLM to use MSCCL++ instead of NCCL for the AllReduce required in tensor parallelism and obtain the time taken for prefill and decode in an offline inference of a batch. The model is distributed over all eight GPUs of a single-node A100-80G machine with tensor parallelism of 8 and CUDA graphs for decodes are enabled for performance. Figure 11 shows that MSCCL++ is on average 1.11x faster than NCCL in decode latency for a range of batch configurations. Here *bsz* denotes the number of batched requests, while *seqlen* denotes the number



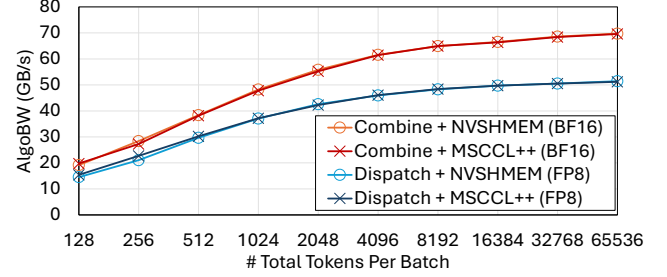
**Figure 12.** DeepSeek-V3 decode throughput using SGLang on two H100 nodes (16 GPUs) with tensor parallelism (1024 input tokens and 1024 output tokens per batch).

of tokens in each request. The reduction in decode time aligns perfectly with what we expect from our standalone AllReduce evaluation in Section 7.1. Since the computation time for prefills is higher than decodes, the communication time improvements with MSCCL++ do not show up prominently, and we see similar or up to 1.06× faster prefill for different batch configurations. A prior work [28] has shown that in production traces, very few active tokens reside in a batch, and for most requests, the majority of end-to-end time is spent in the decode phase. Thus, the performance improvements by MSCCL++ are expected to translate well to real-world workloads.

vLLM also has a custom hand-written AllReduce kernel for a single node. For different message sizes, MSCCL++’s AllReduce performs similar or up to 3× faster than the custom kernel, with a geomean improvement of 1.4×. For end-to-end decode latency, MSCCL++ performs similar or up to 1.11× (on average 1.04×) faster than inference with the custom AllReduce kernel.

**LLM Inference with SGLang.** SGLang [32] is a popular LLM inference framework that has adopted MSCCL++ for accelerating AllReduce. We evaluated end-to-end distributed inference of a recent LLM, DeepSeek-V3 [6], using SGLang on two H100 nodes with tensor parallelism of 16. SGLang has a custom AllReduce kernel but that is limited to a single node. Before adopting MSCCL++, SGLang was either using NCCL or a custom all-reduce kernel (depending on the platform and input size) by default, so that is our baseline. Figure 12 shows the decode throughput of the baseline and MSCCL++, and the speedup in decode throughput due to MSCCL++. MSCCL++ is consistently faster and on average 1.31× faster than NCCL.

**Expert Parallelism with DeepEP.** DeepEP [37] is an implementation of expert parallelism for Mixture-of-Experts (MoE) layers, which is proposed by DeepSeek-V3 [6] for both training and inference. The original code leverages a hardware feature called InfiniBand GPUDirect Async (IBGDA) [15] (featured by NVSHMEM [23]) that implements an InfiniBand networking stack inside GPU, so that GPU can directly communicate with NIC bypassing CPU. However, this feature is



**Figure 13.** DeepEP on two H100 nodes (16 GPUs) with expert parallelism (DeepSeek-V3 settings: hidden size 7168, top-k 8, 256 total experts, FP8 dispatch and BF16 combine).

only available on NVIDIA GPUs combined with Mellanox NICs at the time of writing. To make it portable to other environments, we integrate MSCCL++ into DeepEP to replace the IBGDA-based networking functions into PortChannel API calls, which uses the RDMA networking stack running on CPU instead (see Figure 4).<sup>4</sup> It took a **couple of weeks for one developer** to add this support, but most of that time was spent in understanding DeepEP code. We evaluated DeepEP using its official benchmark and setup. IBGDA was enabled for all experiments, which was leveraged by NVSHMEM, but not by MSCCL++. Figure 13 demonstrates that the two implementations exhibit no noticeable performance difference. Unlike NCCL, MSCCL++ is flexible enough to implement DeepEP-like applications efficiently. The IBGDA stack implementation in DeepEP is less portable and hard to maintain, while MSCCL++’s PortChannel simplifies this code drastically while being portable and performant.

#### 7.4 Portability and Implementation Effort

MSCCL++ is designed to be portable. Code using MSCCL++ collectives, algorithms using the MSCCL++ DSL, and kernels using the MSCCL++ primitives are portable as long as the underlying MSCCL++ Primitive API supports that hardware. Our design makes it easy to add support for new hardware. As an example, MSCCL++ initially supported only NVIDIA GPUs; to add support for AMD MI300x GPUs, our implementation took only **7 weeks for one developer**: 3 weeks for basic AMD GPU support and 4 weeks to develop new AllReduce algorithms that outperform RCCL/MSCCL for message sizes of 1KB - 1GB. We counted the lines of difference between before and after introducing support for the new MI300x architecture: RCCL 35480 vs MSCCL++ 1307 (27× smaller). It is especially remarkable that the AMD-specific code in MSCCL++ (excluding makefiles and algorithms) is fewer than 10 lines of code, while RCCL, the hard-forked NCCL for AMD GPUs, is substantially diverged from NCCL. MSCCL++ could achieve this because the low-level API of AMD GPUs (i.e., HIP) is almost the same as that of NVIDIA

<sup>4</sup>IBGDA can be supported by the MSCCL++ PortChannel API in the future.

GPUs (i.e., CUDA), and the MSCCL++ Primitive API is only a shallow abstraction on top of the low-level API. This significantly reduces the effort to maintain and develop MSCCL++.

The MSCCL++ design makes it easy to extend the Primitive API to support new hardware features. For example, NVIDIA NVLink 4.0 introduced support for multimed instructions for switch-based aggregation and multicast. Since this feature introduced a very different concept from previous interfaces (i.e., PortChannel or MemoryChannel), we developed the SwitchChannel interface to support this feature. The development took only **8 weeks for two developers**, including learning the basic usage of this feature, abstracting the feature as a new type of channel (i.e., SwitchChannel), and finally developing a new AllReduce algorithm using SwitchChannel that outperforms NCCL/MSCCL by more than 2.2× on average for message sizes of 1KB - 1GB.

## 8 Related Work

**GPU Communication Primitives:** NVSHMEM [23] is an extension of the OpenSHMEM [27] API for NVIDIA GPUs for providing a shared-memory abstraction. In contrast, MSCCL++ provides performance-preserving portable abstractions and higher-level DSL for developing efficient communication collective algorithms. Concurrent to our work, NVSHMEM recently introduced support for inter-node communication (after MSCCL++ was open sourced). MSCCL++ provides different interconnect abstractions, namely PortChannel, MemoryChannel, and SwitchChannel, to enable optimized implementations. As an example, NVSHMEM’s put always uses thread-copy when it is feasible, so users cannot choose DMA-copy on NVLink. Similarly, NVSHMEM exposes raw multimed capabilities by exposing underlying memory pointers and users need to implement their kernels using PTX instructions. With an eye towards portability, SwitchChannel abstracts low-level details with a uniform synchronization mechanisms. Finally, NVSHMEM does not provide an API like MemoryChannel’s LL protocol. It may be possible to implement such an API using NVSHMEM, but that would add burden on users.

ARK [11] is another work that proposes a GPU-side control plane for communication, but it is implemented as a monolithic end-to-end ML system rather than a standalone communication library.

**DSL:** Cowan et al. [4] introduced MSCCLang, a DSL to specify collective communication algorithms specialized for a message size and network topology. MSCCLang is built on top of NCCL and RCCL send-recv abstraction, so is limited to two-sided, synchronous communication. Our MSCCL++ DSL provides a one-sided, asynchronous API that enables expressing more efficient communication algorithms. Due to this, MSCCL++ is faster than MSCCL, which is built using MSCCLang DSL, as shown in our evaluation.

**Communication Libraries:** NVIDIA Collective Communications Library (NCCL) [22] and AMD ROCm Collective Communications Library [1] (RCCL) are vendor supplied libraries for NVIDIA and AMD GPUs respectively. We showed that MSCCL++ Collectives perform better than both for AI inference workloads. RCCL has since adopted MSCCL++.

**Communication Algorithms:** SCCL [2], TACCL [33], and TE-CCL [14] aim to accelerate GPU collective communication by synthesizing efficient data transfer algorithms using the NCCL and RCCL send-recv abstraction. Their techniques are orthogonal and their performance may improve if they generate algorithms using the MSCCL++ Primitives API.

**LLM Inference Frameworks:** LLM inference frameworks like SGLang [32], DeepEP [37], vLLM [13], and TensorRT-LLM [25] implement custom kernels for achieving high performance communication. However, the custom implementations are not general-purpose and often only limited to specific interconnects like NVLink or xGMI. MSCCL++ not only performs similar to their custom implementations on their corresponding interconnects, but also supports efficient communication across multiple types of interconnects. SGLang has since adopted MSCCL++.

## 9 Conclusion and Future Work

MSCCL++ is a novel GPU communication stack designed for high-performance AI applications. By exposing the primitive communication functionalities as straightforward user interfaces, MSCCL++ enables fine-grained optimizations for GPU experts, while also providing higher-level interfaces for quick optimizations. Moreover, such a design can reduce the overall development and optimization effort for GPU communication, and accelerates adoption of fast evolving hardware technologies. By implementing collective communication using the proposed MSCCL++ interfaces, we can achieve up to 5.4× speedup for standalone collectives, and up to 1.38× speedup for end-to-end AI inference. MSCCL++ could be used for accelerating AI training workloads too, but we leave that for future work. MSCCL++ could also evolve into a general I/O abstraction for AI accelerators in the future; e.g., by supporting non-GPU accelerators and file system I/O transports like BlitzScale [36].

## References

- [1] AMD. 2025. ROCm Communication Collectives Library (RCCL). <https://github.com/ROCm/rccl>. [Online; accessed Dec 2025].
- [2] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*.
- [3] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: GPU-Efficient Communication Scheduling for Deep Learning Training. In *Proceedings of the 38th ACM Special Interest Group on Data Communication (SIGCOMM)*.

- [4] Meghan Cowan, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, and Yifan Xiong. 2022. MSCCL: Microsoft Collective Communication Library. arXiv:2201.11840
- [5] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [6] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437
- [7] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. 2025. Token-Weave: Efficient Compute-Communication Overlap for Distributed LLM Inference. arXiv:2505.11329
- [8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783
- [9] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoeftler. 2025. Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms. arXiv:2507.04786
- [10] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, HoYuen Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. 2023. Tutel: Adaptive Mixture-of-Experts at Scale. In *Proceedings of the 6th Conference on Machine Learning and Systems (MLSys)*.
- [11] Changho Hwang, Kyoungsoo Park, Ran Shu, Xinyuan Qu, Peng Cheng, and Yongqiang Xiong. 2023. ARK: GPU-driven Code Execution for Distributed Deep Learning. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [12] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*.
- [14] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. 2024. Rethinking Machine Learning Collective Communication as a Multi-Commodity Flow Problem. In *Proceedings of the 38th ACM Special Interest Group on Data Communication (SIGCOMM)*.
- [15] Pak Markthub, Jim Dinan, Sreeram Potluri, and Seth Howell. 2022. Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async>. [Online; accessed Dec 2025].
- [16] Microsoft. 2025. Microsoft Collective Communication Library. <https://github.com/azure/msccl>. [Online; accessed Dec 2025].
- [17] Microsoft. 2025. MSCCL++: A GPU-driven communication stack for scalable AI applications. <https://github.com/microsoft/mscclpp>. [Online; accessed Dec 2025].
- [18] Microsoft. 2025. MSCCL Scheduler. <https://github.com/azure/msccl-scheduler>. [Online; accessed Dec 2025].
- [19] NVIDIA. 2025. 14.9. Multicast Support — CUDA C++ Programming Guide. [docs.nvidia.com/cuda/archive/13.1.0/cuda-c-programming-guide/index.html#multicast-support](https://docs.nvidia.com/cuda/archive/13.1.0/cuda-c-programming-guide/index.html#multicast-support). [Online; accessed Dec 2025].
- [20] NVIDIA. 2025. Multinode NVLink User Guide. <https://docs.nvidia.com/multi-node-nvlink-systems/mnnvlink-user-guide/>. [Online; accessed Dec 2025].
- [21] NVIDIA. 2025. nvbandwidth: A tool for bandwidth measurements on NVIDIA GPUs. <https://github.com/NVIDIA/nvbandwidth>. [Online; accessed Dec 2025].
- [22] NVIDIA. 2025. NVIDIA Collective Communications Library (NCCL). <https://github.com/NVIDIA/nccl>. [Online; accessed Dec 2025].
- [23] NVIDIA. 2025. NVIDIA OpenSHMEM Library (NVSHMEM) Documentation. <https://docs.nvidia.com/nvshmem/api/>. [Online; accessed Dec 2025].
- [24] NVIDIA. 2025. Optimizing for Low-Latency Communication in Inference Workloads with JAX and XLA. [developer.nvidia.com/blog/optimizing-for-low-latency-communication-in-inference-workloads-with-jax-and-xla/](https://developer.nvidia.com/blog/optimizing-for-low-latency-communication-in-inference-workloads-with-jax-and-xla/). [Online; accessed Dec 2025].
- [25] NVIDIA. 2025. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>. [Online; accessed Dec 2025].
- [26] NVIDIA. 2025. User Buffer Registration – NCCL 2.26.2 documentation. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/bufferreg.html>. [Online; accessed Dec 2025].
- [27] OpenSHMEM. 2025. OpenSHMEM Application Programming Interface. <https://github.com/openshmem-org/specification>. [Online; accessed Dec 2025].
- [28] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*.
- [29] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of the 6th Conference on Machine Learning and Systems (MLSys)*.
- [30] Rolf Rabenseifner. 2004. Optimization of Collective Reduction Operations. In *Computational Science - ICCS 2004, 4th International Conference, Kraków, Poland, June 6-9, 2004, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 3036)*. Springer, 1–9.
- [31] Linux RDMA. 2025. perftest: Infiniband Verbs Performance Tests. <https://github.com/linux-rdma/perftest>. [Online; accessed Dec 2025].
- [32] SGLang. 2025. SGLang is a fast serving framework for large language models and vision language models. <https://github.com/sgl-project/sglang>. [Online; accessed Dec 2025].
- [33] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [34] Marc Snir, William Gropp, Steve Otto, Steven Huss-Lederman, Jack Dongarra, and David Walker. 1998. *MPI—the Complete Reference: the MPI core*. Vol. 1. MIT press.
- [35] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. 2023. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [36] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. 2025. BLITZSCALE: fast and live large model autoscaling with O(1) host caching. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation (OSDI)*.
- [37] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>. [Online; accessed Dec 2025].

## A Artifact Appendix

### A.1 Abstract

The artifact includes implementations of the three APIs described in this paper, namely the Primitive API (C++/CUDA) for low-level optimization of GPU communication, the DSL API (Python) that is implemented over the Primitive API and enables high-level description of custom collective communication algorithms, and the Collective API (C++) that implements the NCCL/RCCL API as-is over the Primitive API and the DSL API. The Collective API can be used to reproduce collective communication results in Section 7.1 and 7.2.

### A.2 Artifact check-list (meta-information)

- **Program:** The latest version of `nccl-tests` (or `rccl-tests`). Publicly available and not included.
- **Run-time environment:** Linux; Ubuntu 20.04 or later is recommended. Docker is recommended, otherwise need manual installations of dependencies such as CUDA (or ROCm), Python3, MPI, and optionally OFED for multi-node experiments.
- **Hardware:** 8 datacenter-grade GPUs. Optionally RDMA NICs for multi-node experiments.
- **Execution:** Sole user recommended for best performance.
- **Metrics:** Latency and bandwidth are reported.
- **Output:** Console output with reported metrics.
- **Experiments:** Step-by-step guide in documentation.
- **How much disk space required (approximately)?**: <1GB for the source code and a build; 10GB for a Docker image with all dependencies.
- **How much time is needed to prepare workflow (approximately)?**: 10 minutes
- **How much time is needed to complete experiments (approximately)?**: 1 hour
- **Publicly available?**: Yes
- **Code licenses (if publicly available)?**: MIT License
- **Archived (provide DOI)?**: 10.5281/zenodo.18334822

### A.3 Description

**A.3.1 How to access.** The source code is publicly available on GitHub at <https://github.com/microsoft/mscclpp>. The repository is sized less than 1MB, and a build would be a few hundreds of MB.

**A.3.2 Hardware dependencies.** At least 2 GPUs are needed for functionality tests, and 8 or more are needed for reproducing numbers in Section 7.1 and 7.2. Datacenter-grade GPUs with high-bandwidth interconnects (NVLink or Infinity Fabric) are recommended (see Table 2). Optionally for multi-node experiments, RDMA NICs with InfiniBand or RoCE configurations are required.

**A.3.3 Software dependencies.** CUDA (or ROCm), Python3, and MPI are required for artifact evaluation. Optionally

OFED is required for multi-node experiments. See details in the "Prerequisites" section in `docs/quickstart.md`.

### A.4 Installation

1. Download and run a provided Docker image. This step can be skipped if the host environment is preferred. See "Docker Images" section in `docs/quickstart.md`.
2. Build from the source. See the step-by-step guide in "Install from Source" section in `docs/quickstart.md`. If the environment is not configured for multi-node execution, we recommend to disable InfiniBand/RoCE features via a CMake option `-DMSCCLPP_USE_IB=OFF`.
3. Build and run `unit_tests` and `mp_unit_tests` for functionality testing. See the step-by-step guide in "Unit Tests" section in `docs/quickstart.md`.

### A.5 Evaluation and expected results

For evaluation, install the latest version of `nccl-tests` (for NVIDIA GPUs) or `rccl-tests` (for AMD GPUs), which are publicly available at <https://github.com/NVIDIA/nccl-tests> and <https://github.com/ROCm/rccl-tests>, respectively. MPI should be enabled during their compilation (set environment variable `MPI=1` during make).

After that, follow the "NCCL/RCCL Benchmark over MSCCL++" section in `docs/quickstart.md` to run `nccl-tests` or `rccl-tests` using the MSCCL++ Collective API. The testing binaries will print latency and bandwidth numbers on the console, similar to what presented in Figure 7, 8, 9, and 10, depending on the GPU type and the number of GPUs. Please refer to the `README.md` of `nccl-tests` for detailed usages.

### A.6 Experiment customization

The implementation of MSCCL++ Collective API can be customized using two different interfaces.

- **Using the Primitive API.** Users can write their own GPU communication kernel using the MSCCL++ Primitive API and let the Collective API invoke the custom kernel. We provide a simple example in `docs/guide/customized-algorithm-with-nccl-api.md`.
- **Using the DSL API.** Users can leverage our Python DSL to describe a collective communication algorithm that can be lowered and executed by the Collective API. We provide a simple example in `docs/dsl_quick_start.md`.

### A.7 Notes

All documentation is also publicly available at <https://microsoft.github.io/mscclpp>.

### A.8 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>